

Investigate the stability of the torque (issue #18)

September 2026

1 Short summary

This report checks the torque sampling used for `motor1`. With the current `PointPer = 101` and $N_{\text{per}} = 1/6$, Ansys returns only 18 torque values. The aim is to check whether this is sufficient for `TorAvg` and `TorRippleRms`, and to find a practical value of `PointPer` considering both stability and computation time.

2 Method

For `motor1`, the transient settings are

$$\text{StopTime} = \frac{N_{\text{per}}}{f}, \quad \text{TimeStep} = \frac{1}{f(\text{PointPer} - 1)}.$$

Here, `PointPer` controls the number of sampling points per period, while $N_{\text{per}} = 1/6$ of a period is simulated.

Two fixed Pareto optimal rotor designs were tested, one from `HacklGenerator_SixLambdas` and one from `HacklGenerator_3BrokenLines`. For each design, `PointPer` was varied from 51 to 1001 in steps of 50, corresponding to 10 to 168 returned torque samples. All other settings were kept fixed. `TorAvg` and `TorRippleRms` were evaluated with `analyze_results()`. The elapsed time of each computation was also recorded. The sweep is implemented in `scripts/sensitivity_torque/sensitivity_torque.py`.

3 Results

Figure 1 shows the convergence of these two torque quantities with increasing `PointPer`.

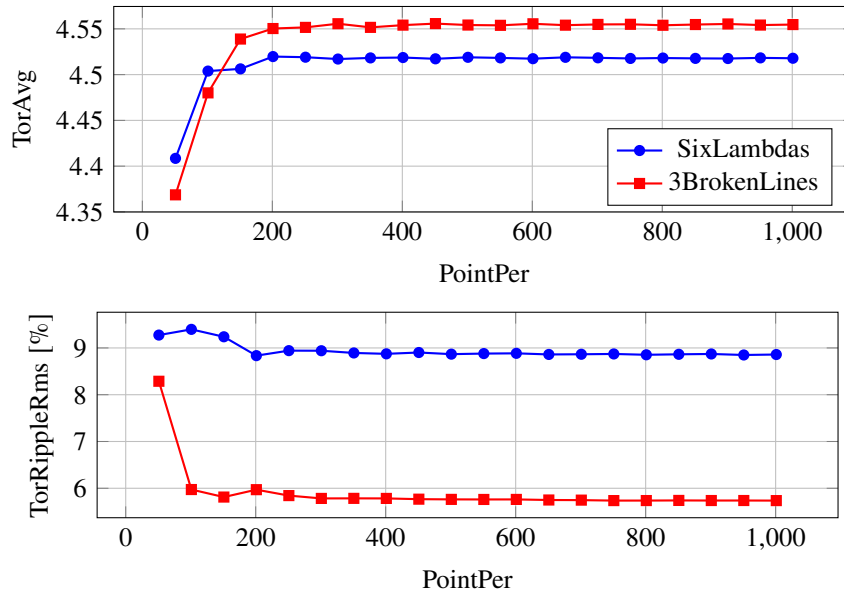


Figure 1: Convergence of mean torque and torque ripple with increasing `PointPer`.

The elapsed computation time is shown in Figure 2. The elapsed computation time increases approximately linearly with `PointPer`.

4 Stability analysis

The results are stable at large `PointPer`, with only small numerical fluctuations. Therefore, the mean of the five largest `PointPer` cases (`PointPer = 801–1001`) was used as the reference value. The relative error is

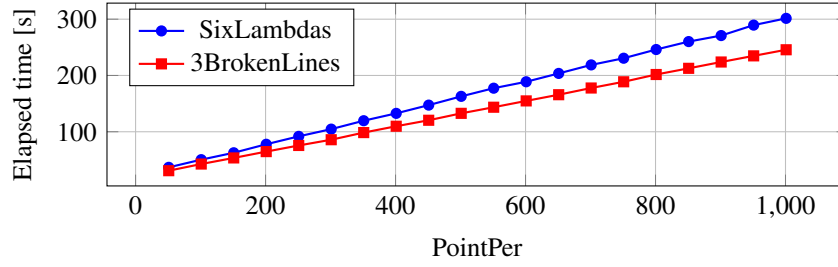


Figure 2: Elapsed computation time with increasing PointPer.

$$\varepsilon_q = \frac{|q - q_{\text{ref}}|}{|q_{\text{ref}}|} \times 100\%.$$

At the current PointPer = 101, the errors in TorAvg and TorRippleRms are 0.31% and 6.07% for SixLambdas, and 1.64% and 4.10% for 3BrokenLines. The ripple is more sensitive to low sampling density.

Both quantities remain within 1% of the reference from PointPer = 301 onward. For a tighter 0.5% limit, PointPer = 501 can be used.

At PointPer = 101, the elapsed times are 50.4 s for SixLambdas and 42.6 s for 3BrokenLines. At PointPer = 301, they increase to 104.7 s and 85.8 s, respectively. This is approximately twice the computation time of the current setting. At PointPer = 501, the elapsed times are 162.9 s and 132.7 s, which is approximately three times the current setting.

5 Recommendation

For motor1, PointPer = 301 is recommended for stable mean torque and torque ripple. Both quantities remain within 1% of the reference, while the elapsed computation time is approximately twice that of the current PointPer = 101.

If an error below 0.5% is required, PointPer = 501 can be used instead, with approximately three times the computation time of the current setting. The current PointPer = 101 is not sufficient for accurate torque ripple.

These results apply to motor1 and the two designs tested here.